NIPPON BUY システム分析設計書

チーム名：Java Cargo ｜ 開発期間：2026.10.13 ～ 2026.11.08 ｜ 対象ソース：kurologi-web

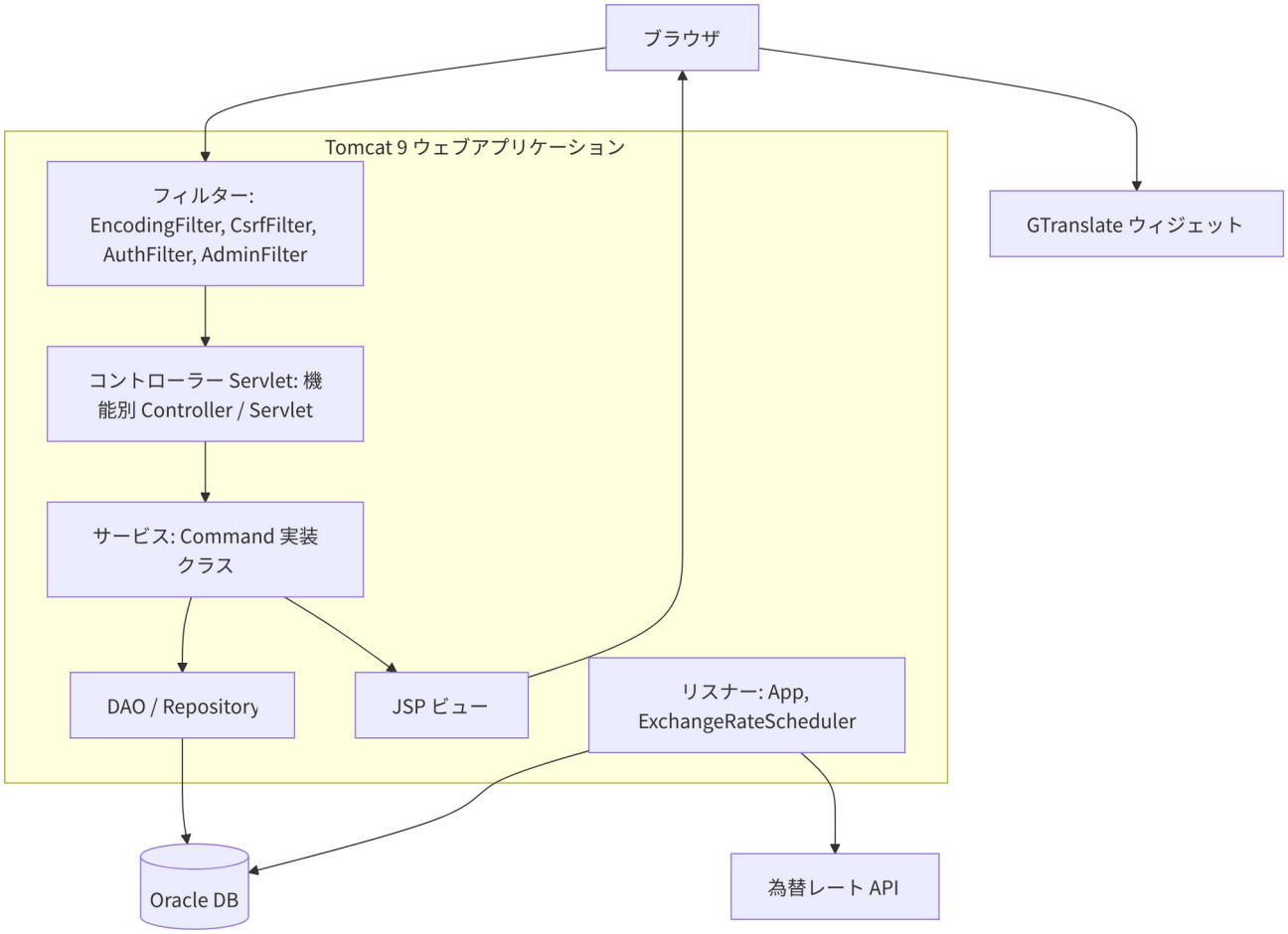
> 添付されたソースを分析して整理しました。ソースで確認できない項目は「要確認」と表記しています。

1. システム概要

NIPPON BUY は JSP/Servlet ベースのウェブアプリケーションで、Oracle DB を使用します。画面（JSP）はスクリプトレット方式で、CSS・JS は各 JSP の中に含まれています。Spring のようなフレームワークは使わず、Servlet、Filter、Listener と、自前で作った Command パターンで構成しました。

項目	内容
言語／ランタイム	Java 17、Tomcat 9（Servlet 4.0、javax.servlet）
画面	JSP（WEB-INF/views 配下、直接アクセス不可）、静的画像は images フォルダー
DB アクセス	JDBC（ojdbc8） + util.DBmanager.getInstance() が Connection を返し、DAO で PreparedStatement を使用
暗号化	jBCrypt（PasswordHasher）
外部連携	open.er-api.com 為替レート API、GTranslate 翻訳ウィジェット
コンテキストパス	/kurologi（開発中は http://localhost:8088/kurologi/ を使用）

2. システム構成図



3. 階層構造とパッケージ

リクエストの流れ：フィルター → コントローラー（アドレス判定）→ Command（実際の処理）→ DAO（DB）→ JSP フォワード（Command が true を返すとフォワード、false なら既にリダイレクト／JSON で応答を終えている）。

パッケージ	役割	主なクラス
jp.kurologi.web	Servlet、フィルター、リスナー、出力ヘルパー	PageServlet、MemberController、MemberFindController、MyPageController、MyPageServlet、AccountServlet、AddressServlet、PaymentServlet、BuyOrderController、ReturnServlet、TrackServlet、CommunityController、AdminCommunityController、AdminOrderController、LoginServlet、LogoutServlet、AuthFilter、AdminFilter、CsrfFilter、EncodingFilter、App、ExchangeRateScheduler、Session、Html、Formats、BaseServlet、MemberBridge

パッケージ	役割	主なクラス
jp.kurologi.service	業務処理（Command インターフェース実装＋ヘルパー）	会員：MemberInsert、IdCheckCommand、LoginCommand、LogoutCommand、FindIdCommand、FindPwCommand、ResetPwCommand、PwQuiz、FindLimiter、MemberService、PasswordHasher、AccountService ／ 注文：BuyOrderForm、BuyOrderInsert、BuyOrderList、BuyOrderView、BuyOrderDelete、BuyOrderLookup、ServiceType、OrderStatus、UploadStore ／ 決済：PaymentService、DepositCharge、ExchangeRateService ／ 追跡・状況：OrderTracker、OrderNotifier、ShipmentBoard、DashboardBoard、MyPageMain ／ コミュニティ：NoticeList、NoticeView、QnaList、QnaView、QnaWrite、QnaUpdate、QnaDelete、MessageList、MessageRead、MessageDelete、FaqList、CommunityPaging、CommunityUtil ／ 管理者：AdminHome、AdminOrderList、AdminOrderAction、AdminOrderTrack、AdminQnaList、AdminQnaAnswer、AdminNotice（List・Write・Update・Delete）、AdminFaq（List・Form・Write・Update・Visible・Delete）、AdminMessage（List・Send）
jp.kurologi.repository	DB アクセス	MemberDAO、MemberFindDAO、DbMemberRepository、BuyOrderDAO、DbPaymentRepository、RefundDAO、TrackingDAO、ExchangeRateDAO、PostDAO、CommentDAO、MessageDAO、FaqDAO、AdminDAO、AdminOrderDAO、AdminQnaDAO、AdminNoticeDAO、AdminFaqDAO、AdminMessageDAO、DaoUtil、InMemoryAddressRepository ほか
jp.kurologi.model	データオブジェクト	MemberDTO、Member、BuyOrderDTO、BuyOrderItemDTO、Payment、Charge、Address、ExchangeRate、Waybill、PostDTO、CommentDTO、MessageDTO、FaqDTO、AdminMessageDTO、ShipmentRow、Page、Grade、MemberType など
util	DB 接続とテスト	DBmanager、DBtest、CommunityDaoTest

3.1 設計上の特徴

- Command インターフェース：boolean doCommand(request, response)。コントローラーは pathInfo を switch で判定し、Command とフォワード先の JSP を選びます。
- App リスナー：サーバー起動時に、会員リポジトリ、決済リポジトリ、住所録リポジトリ、サービスを組み立てて ServletContext に保管します。
- リポジトリの二重構造：会員・決済は DB（DbMemberRepository、DbPaymentRepository）、住所録はメモリ（InMemoryAddressRepository）です。ログイン時に MemberBridge が DB 会員をメモリ会員リポジトリにも登録して既存画面に合わせます（暫定的な便宜策）。
- 新規画面の追加：PageServlet がアドレスと同名の JSP へフォワードするため、JSP を作るだけで開きます（名前は英小文字、数字、_ のみ許可し、パス操作を防止）。

4. モジュール別分析

4.1 会員モジュール

機能	アドレス	コントローラー／Command	DAO	画面
会員登録	/member/join	MemberController / MemberInsert	MemberDAO.memberInsert	join/join.jsp
ID 重複確認	/member/idCheck	IdCheckCommand	MemberDAO.idCheck	（Ajax）
ログイン	/member/login、POST /login	MemberController・LoginServlet / LoginCommand	MemberDAO.login、updateLastLogin	join/login.jsp、ホームのログインボックス
ログアウト	/member/logout、/logout	LogoutCommand、LogoutServlet	-	-
ID 検索	/member/find/id	MemberFindController / FindIdCommand	MemberFindDAO	join/find_id.jsp
パスワード検索	/member/find/pw、/member/find/pwReset	FindPwCommand、ResetPwCommand	MemberFindDAO	join/find_pw.jsp
アカウント管理	/mypage/account、/mypage/account/withdraw	AccountServlet / AccountService	DbMemberRepository	mypage/account.jsp

処理ルール：ID は英数字 4～20 文字、メールアドレス・電話番号は正規表現で検証、パスワードとパスワードクイズの答えは BCrypt でハッシュ化（クイズの答えは空白除去・小文字正規化の後にハッシュ化、最大 20 文字）、検索機能はセッションごとに 5 回失敗すると 10 分間遮断します。

4.2 注文書モジュール（購入代行・配送代行）

機能	アドレス	Command	DAO
注文書入力	/buyorder/form、/shiporder/form	BuyOrderForm	BuyOrderDAO.findById
保存・修正・受付申請	POST /buyorder/save	BuyOrderInsert	BuyOrderDAO.insert / update、DbPaymentRepository.saveCharge
一覧	/buyorder/list	BuyOrderList	BuyOrderDAO.countByMember、listByMember
詳細	/buyorder/view	BuyOrderView	BuyOrderDAO.findById
削除	POST /buyorder/delete	BuyOrderDelete	BuyOrderDAO.delete
住所録・商品・最近の照会	/buyorder/addresses、/products、/recent	BuyOrderLookup	BuyOrderDAO.recentOrders、recentProducts
画像アップロード	/buyorder/upload（Ajax）	UploadStore を使用	ファイルシステム

- ServiceType がリクエストアドレス（/buyorder または /shiporder）で BUY／SHIP を判別します。
- 受付待ち（mode=wait）は氏名と英語商品名のみ検証し、受付申請（mode=submit）は住所・電話・25 項目の同意を検証します。すべての検証はサーバーで再実施し、失敗した場合は入力値を保持したままフォームへ戻します。
- 注文＋商品の保存は 1 トランザクションです。修正・削除は SQL の WHERE で、自分の注文（MEMBER_ID）かつ受付待ち（WAIT）のときのみ許可します。

- 画像はプロジェクトフォルダーの外（ユーザーフォルダー/kurologi-upload/buy）にランダムな 32 桁のファイル名で保存し、決められたファイル名パターンのときだけ開きます。場所は -Dkurologi.upload.dir オプションで変更できます。

4.3 決済モジュール

機能	アドレス	処理
費用決済／決済履歴	GET /mypage/payment（?tab=history）	PaymentService.pendingCharges、history
預託金決済	POST /mypage/payment	PaymentService.pay：金額をサーバーで再計算 → 預託金差し引き → 待ち明細 PAID → Payment 記録 → 送り状割り当て → BUY_ORDER.PAY_STATUS=PAID
試験用チャージ	POST /mypage/payment/charge-deposit	chargeTestDeposit（＋30,000 ウォン）
預託金チャージ申請	/mypage/deposit	DepositCharge：無通帳入金 1,000～1,000,000 ウォン、1,000 ウォン単位、入金者名は必須
為替レート	サーバー起動 10 秒後、以降 1 時間ごと	ExchangeRateScheduler → ExchangeRateService.refreshIfNeeded：今日の為替レートがないときだけ API を呼び出し

為替レートの計算：基準レート（API の市場レート）に、送金レート＋1.0%、告示レート＋2.5% を加算し、決済には告示レートを使用します。注文金額（ウォン）＝円貨合計金額 × 告示レート ÷ 100。直近の為替レートは 10 分間メモリにキャッシュします。

同時実行：会員別ロック（memberLocks）で、預託金の差し引き、待ち明細の状態変更、決済記録を一括処理します。

4.4 管理者の注文管理

機能	アドレス	処理
注文一覧	/admin/orders（svc、st、page フィルター）	AdminOrderList、AdminOrderDAO.counts / list / itemsOf
注文処理	POST /admin/orders/action	AdminOrderAction（action：receive、pay、ship、customs、deliver、delete）→ AdminOrderDAO.advance / deleteOrders → OrderNotifier 通知
配送追跡	GET /admin/orders/track	AdminOrderTrack → OrderTracker.toJson（JSON を直接応答）

- 処理後に戻る一覧アドレスとフィルター・action の値は、ホワイトリストで検証してサーバーで再作成します。
- 出庫は入庫完了＋決済完了のときのみ可能です。削除は決済金額を預託金に返金し、該当する注文・請求・決済をあわせて整理します。
- 通知送信の失敗は、注文状態の変更に影響しません。
- 配送追跡は、実際の宅配会社 API を使わず、送り状番号（PAYMENT）と注文進行段階でイベントを構成します（stage1・stage2 関数を差し替えれば API 連携が可能）。

4.5 送り状照会・現地返品・総合状況

機能	アドレス	処理
韓国送り状照会	GET /track/kr?type=AIR|SEA&no=	TrackServlet → TrackingDAO.findKorean（番号形式は英字・数字・- の 6～30 文字、ログイン不要）
日本送り状照会	GET /track/jp?type=宅配会社&no=	TrackServlet → TrackingDAO.findJapanese

機能	アドレス	処理
現地返品申請一覧	GET /status/return	ReturnServlet（JSON）：配送中の注文＋保有預託金
現地返品キャンセル	POST /status/return/cancel	ReturnServlet → RefundDAO.refund（最大 20 件）
総合状況	/status/dashboard	PageServlet → dashboard.jsp、DashboardBoard が DB を直接照会
マイページの運送品状況	/mypage	ShipmentBoard.load（直近の決済 5 件）

返金（RefundDAO.refund）は、FOR UPDATE で注文・請求をロックした後、請求状態 REFUNDED → 紐づく決済がすべて返金されたら決済状態 REFUNDED → 預託金に加算 → 注文 CANCELED の順に処理します。

4.6 コミュニティモジュール

機能	アドレス	**Command**	**DAO**
お知らせ一覧／詳細	/community/notice、/notice/view	NoticeList、NoticeView	PostDAO
1:1 一覧／詳細	/community/qna、/qna/view	QnaList、QnaView	PostDAO
1:1 作成／修正／削除	/community/qna/write、/update、/delete	QnaWrite、QnaUpdate、QnaDelete	PostDAO
通知一覧／既読／削除	/community/message、/read、/delete	MessageList、MessageRead、MessageDelete	MessageDAO
FAQ	/community/faq	FaqList	FaqDAO
管理者ホーム	/admin/community	AdminHome	AdminDAO.summary
管理者 1:1 一覧・回答	/admin/community/qna、/qna/answer	AdminQnaList、AdminQnaAnswer	AdminQnaDAO
管理者お知らせ	/admin/community/notice、/write、/update、/delete	AdminNotice*	AdminNoticeDAO
管理者 FAQ	/admin/community/faq、/write、/update、/visible、/delete	AdminFaq*	AdminFaqDAO
管理者通知	/admin/community/message、/send	AdminMessageList、AdminMessageSend	AdminMessageDAO

お知らせ・1:1 は COMMUNITY_POST を BOARD_TYPE（NOTICE/QNA）で区別します。ログアウト状態で /community/qna などログインが必要な画面にアクセスすると、ホームのログインボックスへ移動します。1:1 の投稿は、サービス層で作成者を確認し、作成者本人だけが閲覧・修正・削除できるようにしています。

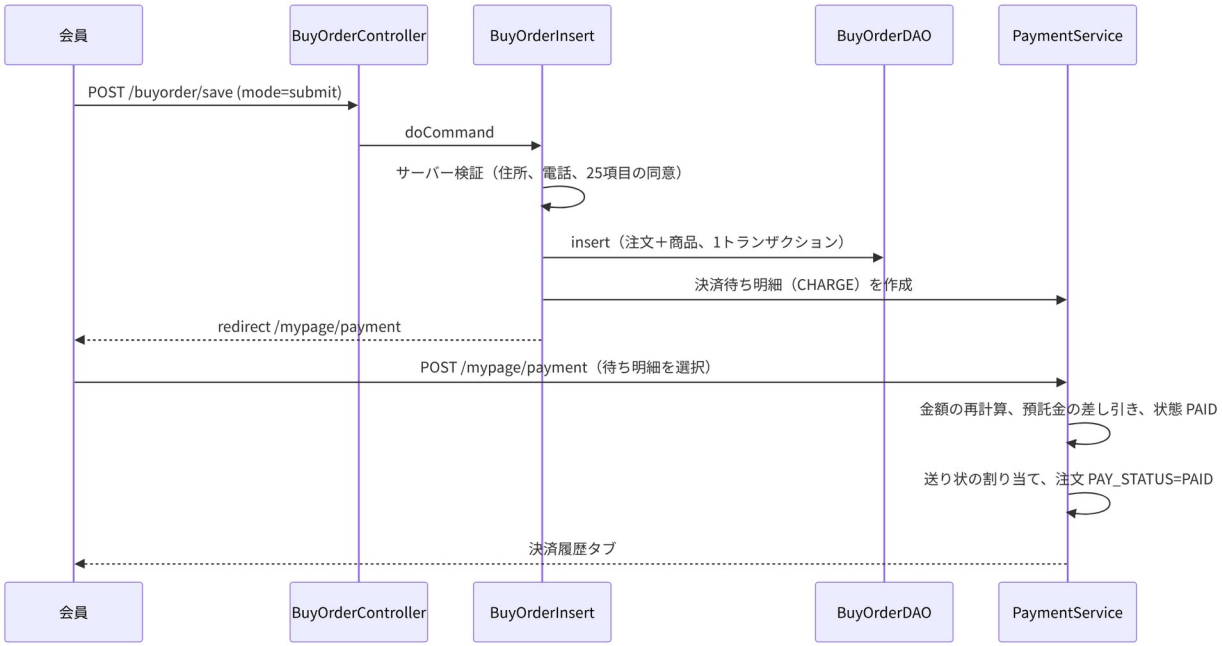
5. 共通処理

区分	クラス	内容
エンコーディング	EncodingFilter（web.xml、最初に実行）	リクエストを UTF-8 に設定
CSRF	CsrfFilter（/*）	POST ごとに hidden の _csrf 値をセッショントークンと比較（MessageDigest.isEqual）、不一致なら 403。ログイン会員の GET でトークンを発行
ログイン検査	AuthFilter（/mypage、/mypage/*）	未ログインなら /?login=required#loginForm へ移動
管理者検査	AdminFilter（/admin、/admin/*）	リクエストごとに DB の ROLE='ADMIN' を確認、一般会員は 404、no-store ヘッダー

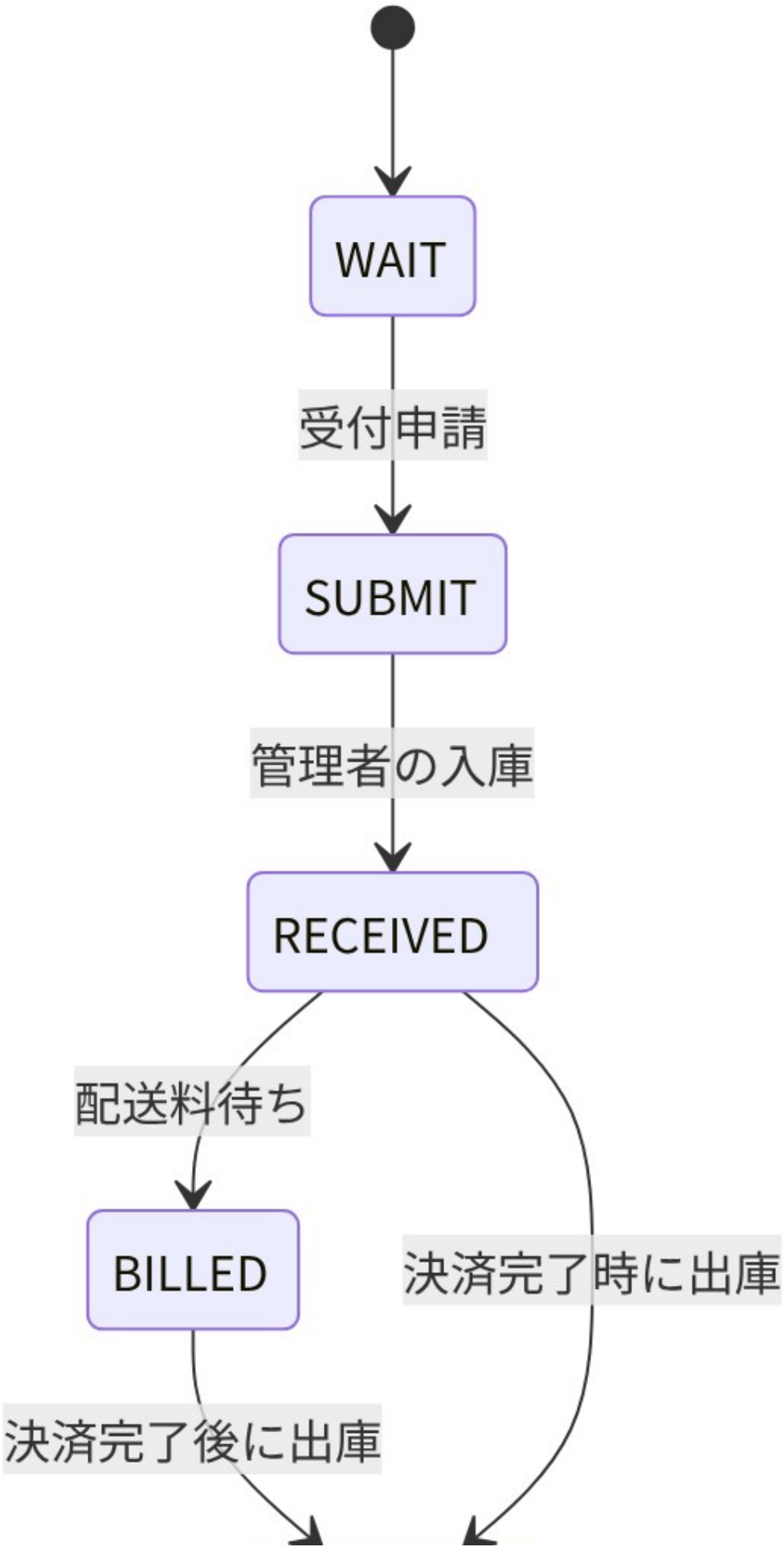
区分	クラス	内容
セッション	Session	memberId、CSRF トークン、1回限りの flash メッセージを管理。セッション 60 分、クッキー http-only、COOKIE 追跡
出力	Html、Formats	Html.esc（HTML エスケープ）、Html.num（千単位カンマ）、日付・金額の書式
共通画面	common/ layout.jsp（part=top、flash、summary、bottom）	ヘッダー・メニュー・フッター・共通スクリプト
多言語	common/translate.jspf	GTranslate（ko、ja、en）、クッキー／localStorage で言語を判定、data-lang-img 画像の差し替え
お気に入り	common/favorite.jspf	お気に入り登録の案内

6. 主要な処理フロー

6.1 受付申請から決済まで



6.2 管理者の段階処理と通知


WAIT
受付申請
SUBMIT
管理者の入庫
RECEIVED
配送料待ち
BILLED
決済完了時に出庫
決済完了後に出庫

段階の変更時に、OrderNotifier が COMMUNITY_MESSAGE へ通知（入庫、決済、出庫、配送）を保存します。02 入庫／03 決済／04 出庫／05・06 配送で、01 申請完了は会員が自分で行ったことなので通知を送りません。

7. 画面アドレスマップ

領域	アドレス	画面（JSP）
ホーム	/	index.jsp
サービス案内	/service/ info_company、info_grade、weightchart2、info_regulation、clearance_policy、listpass_help、manual	service/*.jsp
総合状況	/status/dashboard、tracking	status/*.jsp
会員	/member/login、join、find/id、find/pw	join/*.jsp
マイページ	/mypage、/mypage/ account、payment、address、deposit	mypage/*.jsp
購入代行	/buy/order_form、/buyorder/list、view	buy/*.jsp
配送代行	/ship/order_form、/shiporder/list、view	ship/order_form.jsp → buy 画面を共用
コミュニティ	/community/notice、qna、message、faq	community/*.jsp
管理者	/admin/orders、/admin/community/*	admin/*.jsp

8. セキュリティ設計の要約

脅威	対応
パスワード漏えい	BCrypt ハッシュで保存
CSRF	すべての POST で _csrf トークンを検査
権限昇格	管理者かどうかをリクエストごとに DB で確認し、一般会員には管理者画面の存在を隠す（404）
他人のデータへのアクセス	照会・修正・削除の SQL に MEMBER_ID 条件を含める
金額の改ざん	決済金額はサーバーで待ち明細の合計から再計算
XSS	出力時に Html.esc を使用
パス操作・ファイルアップロード	PageServlet の名前の正規表現、アップロードの拡張子・サイズ・ファイル名パターンの制限
総当たり攻撃	検索機能で 5 回失敗すると 10 分間遮断（セッション基準のため、クッキー削除で回避可能）

9. 限界と改善課題

- 住所録と決済の一部がメモリ保存または暫定構造（MemberBridge）です。DB 移行後に MemberBridge の削除を推奨します。
- 決済・送り状・宅配照会は試験用の実装です（PG、宅配会社 API は未連携）。
- サーバー再起動でメモリ上の住所録が初期化されます。
- 同じクラスが kurologi-web 以外のフォルダー（jsppossibleOnGithubpull、配送追跡_修正ファイル、ルートの sql など）にもコピーとして存在し、最新版の整理が必要です。ルートフォルダーの README は初期の静的 HTML 版の説明です。
- 一部の SQL ファイル（community.sql、Community ddl v2.sql、community_ddl_full.sql）が重複しており、実行順序の整理が必要です（DB 設計書を参照）。
- web.xml の kurologi.devAutoLogin は、運用デプロイ時に必ず false にします。